Actividad: Interfaces Naturales de Usuario UD2

Fabiana Sotillo - DAM2

Ejercicio 1. ¿Modelo o regla? Decidir con criterio

a) Para cada caso, indica si está justificado usar aprendizaje automático o basta con una regla programada. Justifica tu respuesta con el criterio visto en clase: ¿una regla manual sería insuficiente, frágil o imposible de mantener?

Caso	Decisión y justificación
A. Comprobar que el DNI escrito en un formulario tiene 8 números y una letra.	Basta una regla programada: comprobar ocho dígitos y una letra es una condición concreta, determinista y fácil de mantener.
B. Reconocer la orden «¡para!» dicha en voz alta por operarios en un taller con maquinaria ruidosa.	Está justificado usar aprendizaje automático porque cambian las voces, acentos, pronunciaciones y condiciones acústicas. Una regla basada en una coincidencia exacta sería frágil e insuficiente. Al ser una orden de seguridad, debe probarse rigurosamente y no ser el único mecanismo para detener maquinaria peligrosa.
C. Detectar si el usuario escribe la palabra «salir» en un campo de comandos de texto.	Basta una regla si se busca un comando textual definido. Puede comprobarse la presencia de la palabra mediante una condición programada.
D. Reconocer qué ejercicio de mano está haciendo un paciente en una app de rehabilitación con la cámara del móvil.	El aprendizaje automático está justificado por la variación de posturas, personas, ángulos e iluminación. Describir manualmente todas las posibilidades sería frágil y difícil de mantener.

Las reglas son apropiadas cuando las condiciones son claras y previsibles. El aprendizaje automático es útil cuando hay variabilidad y resulta difícil enumerar manualmente todos los casos.

b) Una app de banca integra un reconocedor de voz ya entrenado. Recibe dos órdenes, ambas con una confianza de 0,62: «transfiere 500 euros a Laura» y «abre la ayuda». Explica qué debería hacer la interfaz en cada caso y por qué no deben tratarse igual. ***(1 punto)***

“Transfiere 500 euros a Laura”: debe pedir confirmación explícita antes de ejecutar la transferencia, porque un error puede causar un perjuicio económico. “Abre la ayuda”: podría abrirla directamente si es una acción de bajo riesgo y reversible; si hay ambigüedad, debe pedir aclaración. No se tratan igual porque la respuesta depende tanto de la confianza como de las consecuencias de equivocarse. Un valor de confianza no garantiza por sí solo que la interpretación sea correcta.

c) Para el caso B, enumera cuatro condiciones reales con las que habría que probar el modelo antes de desplegarlo y explica por qué no basta con que funcione en el portátil del desarrollador

1. Niveles de ruido: probar con maquinaria apagada y en funcionamiento, porque el ruido puede enmascarar la orden.
2. Voces y acentos distintos: incluir diferentes tonos, velocidades, pronunciaciones y volúmenes.
3. Distancias y orientaciones: probar cerca y lejos del micrófono y desde diferentes posiciones.

4. Palabras parecidas y conversaciones de fondo: medir tanto órdenes no detectadas como falsas detecciones.

No basta con el portátil del desarrollador porque el micrófono, el entorno acústico y las condiciones reales pueden ser diferentes.
El sistema debe evaluarse en escenarios representativos antes de desplegarse.

Ejercicio 2.

a) Sin ejecutarlo, indica qué devuelve classify para cada una de estas entradas y explica el motivo: «Abrir Camara», « ABRIR la cámara », «activa la ayuda por favor», «necesito ayuda». ¿Qué ocurre si recibe null?

Entrada	Resultado	Motivo
«Abrir Camara»	OPEN_CAMERA	Tras pasar a minúsculas, contiene «abrir» y «camara».
« ABRIR la cámara »	UNKNOWN_INTENT	Se recortan los espacios y se pasa a minúsculas, pero no se eliminan tildes; «cámara» no coincide con «camara».
«activa la ayuda por favor»	UNKNOWN_INTENT	La condición exige «activar» y «ayuda». «activa» no contiene la secuencia «activar».
«necesito ayuda»	UNKNOWN_INTENT	Contiene «ayuda», pero no «activar».
null	NullPointerException	Se llama a toLowerCase() antes de comprobar si text es null.

c) Escribe un método main que pruebe tu clase con al menos seis entradas (incluidas null, una cadena en blanco y una frase con tildes) y presenta los resultados en una tabla con las columnas: entrada, intención esperada, intención obtenida.

Entrada	Esperada	Obtenida
«Abre la cámara, por favor»	OPEN_CAMERA	OPEN_CAMERA
null	EMPTY_INPUT	EMPTY_INPUT
« »	EMPTY_INPUT	EMPTY_INPUT
«ACTIVA la ayuda, por favor»	ENABLE_HELP_MODE	ENABLE_HELP_MODE
«Quiero salir de la aplicación»	CLOSE_APP	CLOSE_APP
«enciende la camára»	OPEN_CAMERA	OPEN_CAMERA
«¿Qué tiempo hace hoy?»	UNKNOWN	UNKNOWN
«no quiero salir»	UNKNOWN (negación)	CLOSE_APP

d) Busca al menos una frase razonable que tu intérprete clasifique mal y explica por qué falla. Después explica, en 3–5 líneas, por qué la pantalla de la aplicación no tendría que cambiar si mañana sustituyes tu clase por un modelo de PLN.

Una frase que se clasifica mal es «no quiero salir». El intérprete devuelve CLOSE_APP porque busca la cadena «salir» sin analizar la negación ni la intención completa. Es un clasificador basado en reglas y coincidencias de palabras, por lo que no comprende el contexto.

La pantalla debe depender del contrato CommandInterpreter y de las intenciones que este devuelve, no de los detalles internos de la implementación. Si se sustituye la clase por un modelo de procesamiento del lenguaje natural que mantenga el mismo contrato, la interfaz puede seguir llamando a detectIntent y reaccionar a los mismos identificadores. Así se separa la presentación de la lógica de interpretación. Solo sería necesario cambiar la interfaz si cambiara el contrato o las intenciones devueltas.

Ejercicio 3. Entrenar, validar y no engañarse

a) Completa la tabla aplicando exactamente ese cálculo e indica si algún conjunto queda vacío.

Se aplica exactamente: trainSize = round(n × 0,6), validationSize = round(n × 0,2) y testSize = n − trainSize − validationSize. Para estos valores positivos, Math.round redondea al entero más cercano.

N.º ejemplos	Entrenamiento	Validación	Prueba	¿Algún conjunto vacío?
2	1	0	1	Sí: validación
6	4	1	1	No
9	5	2	2	No
12	7	2	3	No
25	15	5	5	No

Para n = 2, trainSize = round(1,2) = 1 y validationSize = round(0,4) = 0; por tanto, entrenamiento tiene 1 ejemplo, validación 0 y prueba 1. La validación queda vacía.

b) Las frases de ejemplo vienen ordenadas por intención: primero todas las de «abrir», después las de «cerrar» y al final las de «ayuda». ¿Qué pasaría si eliminas la línea Collections.shuffle? ¿Para qué sirve el 42?

Si se elimina Collections.shuffle, los ejemplos seguirían agrupados por intención: el entrenamiento podría contener sobre todo ejemplos de «abrir», la validación de «cerrar» y la prueba de «ayuda». Las particiones no serían representativas y la evaluación podría ser engañosa. Random(42) fija una semilla para que la mezcla sea reproducible: con la misma lista y el mismo entorno se obtiene la misma secuencia pseudoaleatoria.

c) ¿Por qué cada subList se envuelve en new ArrayList<>(...)? ¿Qué «fallo silencioso» evita añadir una comprobación de que ningún conjunto está vacío?

subList devuelve una vista respaldada por la lista original. Envolverla en new ArrayList<>(...) crea una lista independiente y evita efectos inesperados si se modifica estructuralmente la lista original, como cambios reflejados en la vista o
ConcurrentModificationException. Comprobar que ningún conjunto está vacío evita continuar silenciosamente con una fase de entrenamiento o evaluación sin ejemplos, lo que puede producir errores o métricas sin significado.

d) Un compañero te dice: «Mi modelo de gestos acierta el 99 % con los datos de entrenamiento y el 61 % con los de validación, así que ya está listo». ¿Qué le responderías? Usa el término técnico adecuado.

Hay indicios de sobreajuste (overfitting): el modelo parece aprender muy bien los ejemplos de entrenamiento, pero generaliza bastante peor a datos no usados para entrenarlo. No está listo solo por obtener un 99 % en entrenamiento. Deben analizarse la diferencia y la calidad y representatividad de los datos; se puede ajustar el modelo usando validación y reservar un conjunto de prueba independiente para la evaluación final.

Nota de entrega: el enunciado solicita que el PDF incluya los ejercicios 1 y 3 y los apartados 2a, 2c y 2d. Los archivos Java se entregan aparte.